

# Relatório de Auditoria de Segurança

**FuuDelivery**

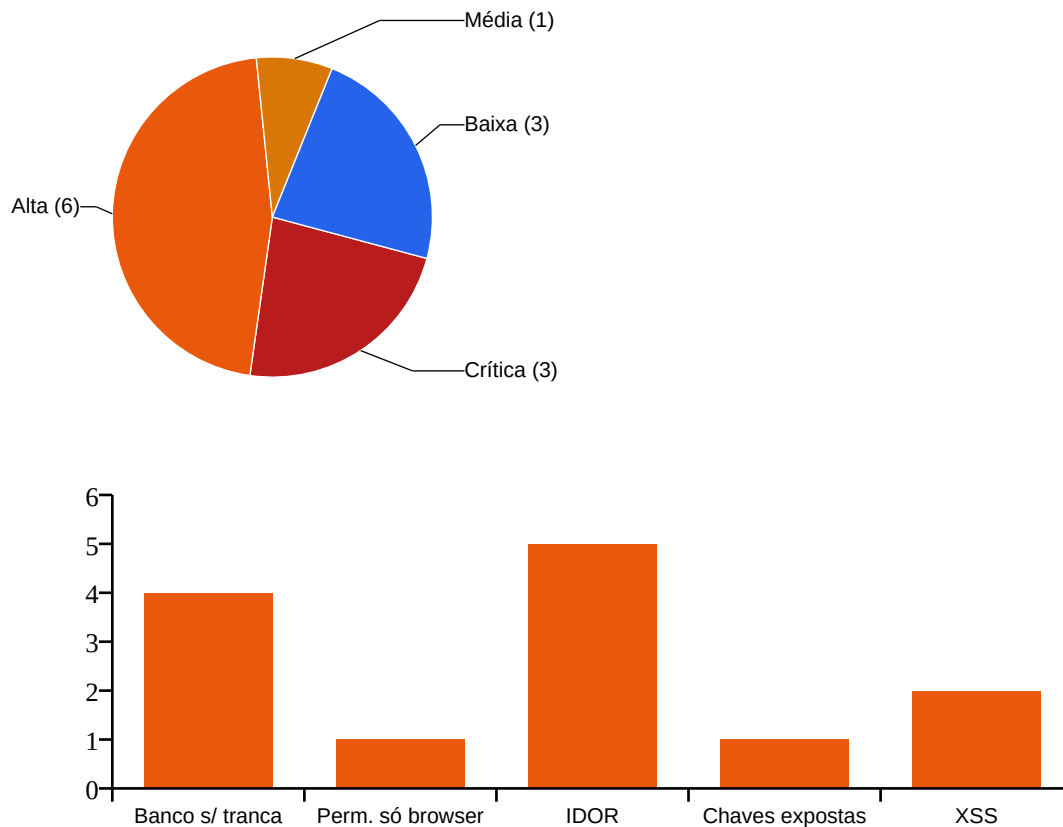
Data: 29/08/2026

Escopo: Backend Go (Fiber/GORM), Frontends React (WebRestaurant, WebAdmin), Deploy Render

**Metodologia:** Cada categoria de segurança foi mapeada para a stack detectada — isolamento de dados via queries GORM com filtro manual por establishment\_id/user\_id (ausência de RLS no Supabase, projeto usa GORM direto); permissões via middleware Go (adminRequired, protectedRoute, canActOnEstablishment); IDOR via varredura sistemática de handlers com parâmetros de ID; chaves via grep de hardcoded values; XSS via análise de JSX e sanitize libraries.

# 1. Resumo Executivo

A auditoria identificou **13 achados** de segurança: **3 críticos**, **6 altos**, **1 médios**, **3 baixos**. **14 pontos fortes** verificados.



## 2. Pontos Fortes Verificados

### ■ Gateway Router com fallback — pkg/gateway/router.go

Router.Select() implementa fallback automático Pagar.me → Asaas → AbacatePay com circuit breaker

### ■ Carteira com SELECT FOR UPDATE — Backend/payment\_api/app/handlers/wallet.go

Operações financeiras usam transação + FOR UPDATE para prevenir race conditions

### ■ Webhook HMAC fail-closed — Backend/payment\_api/app/handlers/webhook.go

Sem ABACATE\_PAY\_WEBHOOK\_SECRET em produção → rejeita (antes só logava)

### ■ CSRF Protection — cmd/fuudelivery/main.go:1530-1540

Middleware valida X-CSRF-Token em todas as mutações POST/PUT/DELETE

### ■ Rate Limiting — cmd/fuudelivery/main.go

Login 10/min, refresh 30/min, mutations protegidas com rate limiter

### ■ Upload de imagem validado — cmd/fuudelivery/pkg/upload/upload.go

Magic bytes + bloqueio SVG + ownership check por entidade

### ■ IDOR em UpdateProduct — Backend/orders\_api/app/handlers/products.go

canActOnEstablishment() antes de permitir alteração

■ **IDOR em DeleteProduct** — Backend/orders\_api/app/handlers/products.go

canActOnEstablishment() antes de permitir exclusão

■ **IDOR em UpdateOrderStatus** — Backend/orders\_api/app/handlers/orders.go:246

canActOnEstablishment(doc.EstablishmentID) antes de mudar status

■ **WebSocket com ticket** — cmd/fuudelivery/main.go:806-870

resolveWSTicket() + wsCanAccessOrder() antes de conexão

■ **Startup validation** — cmd/fuudelivery/main.go:1410-1440

Fatality se JWT\_SECRET ou DB\_CONNECTION\_STRING ausentes em produção

■ **Sessão HttpOnly** — Backend/auth\_api/app/handlers/session\_handler.go

Cookies com HttpOnly + Secure + SameSite=None

■ **EstablishmentWithdraw com ownership** — Backend/payment\_api/app/handlers/wallet.go

GetEstablishmentIDFromToken() + ownership check

■ **TopUp/Deduct com ownership** — Backend/payment\_api/app/handlers/wallet.go

tokenUserID != req.UserID → rejeita

## 3. Achados Detalhados

### 3.1 Banco sem tranca

#### [Crítica] GetUser sem verificação de posse

Backend/auth\_api/app/handlers/user\_handler.go:22-32

Qualquer usuário autenticado pode ler os dados completos de QUALQUER outro usuário (nome, email, telefone, role, establishment\_id) simplesmente passando o ID na URL.

```
userID := c.Params("id") var user models.User models.DB.First(&user, userID) return c.JSON(user) // retorna TUDO, sem checagem
```

**Exploração:** IDOR clássico: GET /users/1, GET /users/2, etc. Um restaurante pode descobrir dados de outros restaurantes ou clientes.

#### [Crítica] ListOrdersByEstablishmentID sem isolamento

Backend/orders\_api/app/handlers/orders.go:363-385

Qualquer usuário autenticado pode listar todos os pedidos de qualquer estabelecimento, incluindo dados de clientes, valores, telefones.

```
establishmentID := c.Params("establishmentId") models.DB.Where("establishment_id = ?", establishmentIDInt).Order("created_at desc").Limit(500).Find(&docs) // Sem canActOnEstablishment()!
```

**Exploração:** GET /orders/1 (estabelecimento do dono) funciona — mas GET /orders/2 (estabelecimento concorrente) também funciona. Vazamento massivo de dados.

#### [Alta] ListCoupons sem isolamento de estabelecimento

Backend/orders\_api/app/handlers/coupon.go:6-18

Lista cupons sem filtrar pelo establishment\_id do token. O parâmetro é query param, não do token.

```
func ListCoupons(c *fiber.Ctx) error { establishmentID := c.Query("establishment_id") query := models.DB if establishmentID != "" { query = query.Where("establishment_id = ? OR establishment_id = 0", establishmentID) } query.Find(&coupons) // qualquer establishment_id funciona
```

**Exploração:** GET /coupons?establishment\_id=1 — qualquer autenticado pode listar cupons de qualquer restaurante.

#### [Alta] GetUserByEstablishment sem verificação de posse

Backend/auth\_api/app/handlers/establishment\_handler.go:6-20

Lista todos os usuários (nome, email, id) de qualquer estabelecimento sem verificar se o chamador pertence ao estabelecimento.

```
func GetUserByEstablishment(c *fiber.Ctx) error { establishmentID, _ := c.ParamsInt("id") models.DB.Where(&models.User{EstablishmentID: uint(establishmentID)}).Find(&user) return c.JSON(user) // lista TODOS os users do estabelecimento
```

**Exploração:** GET /establishments/1/users — qualquer autenticado pode listar todos os funcionários de qualquer restaurante.

### 3.2 Permissão só no navegador

#### [Média] WebAdmin ProtectedRoute não verifica role admin

Frontend/WebAdmin/src/App.js:47-57

O componente `ProtectedRoute` do `WebAdmin` apenas checa se o usuário está logado (`user != null`), mas NÃO verifica se é admin. Qualquer role (`restaurant`, `client`, `delivery`) pode acessar o `WebAdmin`.

```
const ProtectedRoute = ({ children }) => { const { user, loading } = useAuth(); if (!user) return <Navigate to="/login" replace />; return children; // sem checagem de role! };
```

**Exploração:** Um restaurante pode acessar `WebAdmin > Gerenciamento de Usuários, ESTABELECIMENTOS`, etc. O backend bloqueia com `adminRequired`, mas a UI fica visível.

## 3.3 IDOR

### [Crítica] DeleteCoupon sem verificação de posse

Backend/orders\_api/app/handlers/coupon.go:24-42

Qualquer usuário autenticado pode desativar cupons de qualquer estabelecimento.

```
func DeleteCoupon(c *fiber.Ctx) error { id := c.Params("id") models.DB.First(&coupon, id) coupon.IsActive = false models.DB.Save(&coupon) // sem checar se pertence ao establishment do token }
```

**Exploração:** DELETE /coupons/999 — restaurante A pode desativar cupons do restaurante B.

### [Alta] GetPaymentByOrder sem verificação de posse

Backend/payment\_api/app/handlers/order\_total.go:6-22

Qualquer usuário autenticado pode consultar detalhes de pagamento de qualquer pedido.

```
func GetPaymentByOrder(c *fiber.Ctx) error { orderID := c.Params("order_id") models.DB.Where("order_id = ?", orderID).First(&payment) return c.JSON(payment) // sem checar ownership }
```

**Exploração:** GET /payments/order/abc123 — qualquer autenticado pode ver valor, método e status de pagamento de qualquer pedido.

### [Alta] HandlerEstablishmentStatus sem verificação de posse

Backend/auth\_api/app/handlers/establishment\_handler.go:28-40

Qualquer usuário autenticado pode abrir/fechar qualquer restaurante, alterando o `OpenData`.

```
func HandlerEstablishmentStatus(c *fiber.Ctx) error { establishmentID := c.Params("id") models.DB.First(&establishment, establishmentID) // toggle OpenData sem canActOnEstablishment() }
```

**Exploração:** PUT /establishments/status/handler/2 — restaurante A pode fechar o restaurante B.

### [Alta] UpsertBusinessHours sem verificação de posse

Backend/auth\_api/app/handlers/business\_hours.go:6-35

Qualquer usuário autenticado pode alterar horários de funcionamento de qualquer restaurante passando `establishment_id` no body.

```
func UpsertBusinessHours(c *fiber.Ctx) error { hours := models.BusinessHours{ EstablishmentID: req.EtablissementID, // do BODY, não do token! DayOfWeek: req.DayOfWeek, IsOpen: req.IsOpen, } // sem canActOnEstablishment(req.EtablissementID) }
```

**Exploração:** POST /establishments/hours com `establishment_id=2` — restaurante A altera horários do restaurante B.

### [Alta] GetCoupon sem verificação de posse

Backend/orders\_api/app/handlers/coupon.go:20-23

Qualquer usuário autenticado pode ler detalhes de cupons de qualquer restaurante.

```
func GetCoupon(c *fiber.Ctx) error { id := c.Params("id") models.DB.First(&coupon, id) return c.JSON(coupon) // sem ownership check
```

**Exploração:** GET /coupons/5 — qualquer autenticado pode ver código, desconto e regras de cupons alheios.

## 3.4 Chaves expostas

### [Baixa] Senha mínima de 6 caracteres

Backend/auth\_api/app/handlers/\*.go:múltiplos

Todas as rotas de criação de conta e reset de senha aceitam senhas de apenas 6 caracteres, abaixo do mínimo recomendado (8+).

```
if len(req.Password) < 6 { return c.Status(400).JSON(fiber.Map{"error": "..."}) }
```

**Exploração:** Contas com senhas fracas como '123456' ou 'abcdef' são vulneráveis a brute force.

## 3.5 XSS

### [Baixa] src={item?.Image} renderiza URL do usuário sem validação

Frontend/WebRestaurant/src/components/CardapioList.js:30

A URL da imagem do cardapio é renderizada diretamente no atributo src sem validação de protocolo. Embora browsers modernos bloqueiem javascript: em src, URLs como data: com HTML embutido podem causar problemas.

```
&lt;img src={item?.Image} /&gt;
```

**Exploração:** Se o backend não validar o protocolo da URL, um atacante pode enviar data:text/html,... como URL de imagem.

### [Baixa] Sem biblioteca de sanitização (DOMPurify ou similar)

Frontend/WebRestaurant/src/:global

O frontend não usa nenhuma biblioteca de sanitização de HTML. Embora não haja dangerouslySetInnerHTML, a ausência de sanitização é um risco se o projeto crescer e começar a renderizar HTML do servidor.

```
// Nenhum import de DOMPurify, xss, ou sanitize-html encontrado
```

**Exploração:** Risco latent: se qualquer componente futuro usar dangerouslySetInnerHTML, não há barreira de segurança.

## 4. Recomendações Priorizadas

**[P1]** Corrigir IDOR em GetUser, ListOrdersByEstablishmentID, GetCoupon, DeleteCoupon, GetPaymentByOrder, GetUserByEstablishment, HandlerEstablishmentStatus, UpsertBusinessHours, BulkUpdateBusinessHours — adicionar canActOnEstablishment() ou equivalente em TODOS os handlers que acessam dados por ID

**[P1]** Adicionar ownership check em ListCoupons e ListCoupons filtrar automaticamente pelo establishment\_id do token, não por query param

**[P2]** WebAdmin ProtectedRoute deve verificar role === 'admin' antes de renderizar children

**[P2]** Aumentar senha mínima para 8 caracteres + complexidade (maiúscula, número, especial)

**[P3]** Validar protocolo de URLs de imagem (permitir apenas https:// e data:image/) no frontend

**[P3]** Adicionar DOMPurify ou equivalente para sanitização defensiva no frontend

## 5. Issues para o GitHub

```
--- ISSUE 1 ---
**[Segurança] IDOR em múltiplos handlers – dados de outros estabelecimentos acessíveis**

**Labels:** `security` `priority:critical`

**Descrição:**
Múltiplos handlers não verificam se o objeto pertence ao establishment_id do usuário autenticado.
Qualquer usuário comum pode ler, alterar ou deletar recursos de outros restaurantes.

**Evidência:**
- `Backend/auth_api/app/handlers/user_handler.go:22-32` – `GetUser` retorna dados completos de
qualquer usuário por ID
- `Backend/orders_api/app/handlers/orders.go:363-385` – `ListOrdersByEstablishmentID` lista
pedidos de qualquer estabelecimento
- `Backend/orders_api/app/handlers/coupon.go:6-42` – `ListCoupons`, `GetCoupon`, `DeleteCoupon`
sem ownership
- `Backend/payment_api/app/handlers/order_total.go:6-22` – `GetPaymentByOrder` expõe dados
financeiros
- `Backend/auth_api/app/handlers/establishment_handler.go:6-20` – `GetUserByEstablishment` lista
funcionários
- `Backend/auth_api/app/handlers/establishment_handler.go:28-40` – `HandlerEstablishmentStatus`
altera status
- `Backend/auth_api/app/handlers/business_hours.go:6-35` – `UpsertBusinessHours` altera horários

**Impacto:**
Vazamento de dados pessoais, manipulação de pedidos, cupons e configurações de restaurantes
concorrentes.

**Sugestão de correção:**
Adicionar `canActOnEstablishment(c, establishmentID)` ou equivalente em TODOS os handlers que
acessam dados por ID de estabelecimento. Para GetUser, restringir a: próprio perfil ou admin.

**Critérios de aceite:**
- [ ] Todos os handlers de leitura por ID verificam posse
- [ ] Todos os handlers de escrita verificam posse
- [ ] Testes unitários provam que usuários não podem acessar dados alheios
--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
**[Segurança] WebAdmin não verifica role admin no frontend**
```

**\*\*Labels:\*\*** `security` `priority:medium`

**\*\*Descrição:\*\***

O componente `ProtectedRoute` em `Frontend/WebAdmin/src/App.js:47-57` apenas verifica se o usuário está logado, sem checar se é admin. Qualquer role pode acessar o WebAdmin.

**\*\*Evidência:\*\***

```
```javascript
const ProtectedRoute = ({ children }) => {
  const { user, loading } = useAuth();
  if (!user) return ;
  return children; // sem checagem de role!
};
```
```

**\*\*Impacto:\*\***

Interface de admin visível para restaurantes/clientes/entregadores, mesmo que o backend bloqueie as chamadas API.

**\*\*Sugestão de correção:\*\***

Adicionar `if (user?.role !== 'admin') return ;` no ProtectedRoute.

**\*\*Critérios de aceite:\*\***

- [ ] WebAdmin ProtectedRoute verifica `user.role === 'admin'`
  - [ ] Usuários não-admin são redirecionados para página apropriada
- FIM ISSUE 2 ---

--- ISSUE 3 ---

**\*\*[Segurança] Senha mínima de 6 caracteres é fraca\*\***

**\*\*Labels:\*\*** `security` `priority:low`

**\*\*Descrição:\*\***

Todas as rotas de criação de conta e reset de senha aceitam senhas de apenas 6 caracteres, abaixo do mínimo NIST (8+).

**\*\*Evidência:\*\***

```
`Backend/auth_api/app/handlers/user_handler.go:329`, `client_handler.go:35`,
`deliveryman_handler.go:146`, `establishment_handler.go:39`, `password_reset.go:24`
```

**\*\*Impacto:\*\***

Contas com senhas fracas são vulneráveis a brute force e credential stuffing.

**\*\*Sugestão de correção:\*\***

Aumentar mínimo para 8 caracteres + exigir pelo menos 1 maiúscula, 1 número.

**\*\*Critérios de aceite:\*\***

- [ ] Senha mínima >= 8 caracteres
  - [ ] Exigência de complexidade documentada na UI
- FIM ISSUE 3 ---

# Anexo — Stack Detectada

**Backend:** Go 1.25, Fiber v2.52, GORM v1.31 (PostgreSQL via Supabase), golang-jwt/v5 (HS256), Redis Streams

**Frontend:** React 19, Axios, Vite, React Router v6, react-toastify

**Mobile:** React Native + Expo (3 apps: AppComida, AppEntrega, AppRestaurante)

**Deploy:** Render (render.yaml), Dockerfile existente

**CI/CD:** GitHub Actions (15+ jobs)

**Database:** PostgreSQL (Supabase), Redis externo (\*.db.redis.io)

**Auth:** JWT HS256 (15min) + Refresh Token (30d) + CSRF Token + HttpOnly Cookies

**Pagamentos:** Multi-gateway (Pagar.me, Asaas, AbacatePay, Mercado Pago) com Router + Circuit Breaker

**Isolamento de dados:** Filtro manual por establishment\_id/user\_id em queries GORM (sem RLS no banco)